

Assignment Number: 3**Name:** Atharv Santosh Danave**Roll No.:** 11**Assignment Title:** Implement Min, Max, Sum and Average operations using Parallel Reduction.**CODE**

```
// =====  
  
// Assignment 3: Parallel Reduction (Min, Max, Sum, Average)  
// Using OpenMP reduction clause  
// =====  
  
#include <iostream>  
#include <vector>  
#include <climits>  
#include <cfloat>  
#include <omp.h>  
#include <chrono>  
#include <random>  
using namespace std;  
  
// -----  
// SEQUENTIAL implementations  
// -----  
  
int seq_min(const vector<int>& arr) {  
    int mn = INT_MAX;  
    for (int x : arr) mn = min(mn, x);  
    return mn;  
}  
  
int seq_max(const vector<int>& arr) {  
    int mx = INT_MIN;  
    for (int x : arr) mx = max(mx, x);  
    return mx;  
}  
  
long long seq_sum(const vector<int>& arr) {  
    long long s = 0;  
    for (int x : arr) s += x;
```

```

    return s;
}

double seq_avg(const vector<int>& arr) {
    return (double)seq_sum(arr) / arr.size();
}

// -----
// PARALLEL implementations using OpenMP reduction
// -----

int par_min(const vector<int>& arr) {
    int mn = INT_MAX;
    #pragma omp parallel for reduction(min:mn) schedule(static)
    for (int i = 0; i < (int)arr.size(); i++)
        mn = min(mn, arr[i]);
    return mn;
}

int par_max(const vector<int>& arr) {
    int mx = INT_MIN;
    #pragma omp parallel for reduction(max:mx) schedule(static)
    for (int i = 0; i < (int)arr.size(); i++)
        mx = max(mx, arr[i]);
    return mx;
}

long long par_sum(const vector<int>& arr) {
    long long s = 0;
    #pragma omp parallel for reduction(+:s) schedule(static)
    for (int i = 0; i < (int)arr.size(); i++)
        s += arr[i];
    return s;
}

double par_avg(const vector<int>& arr) {
    return (double)par_sum(arr) / arr.size();
}

```

```

}

// -----
// HELPER
// -----

template<typename Func>
double timeIt(Func f) {
    auto s = chrono::high_resolution_clock::now();
    f();
    auto e = chrono::high_resolution_clock::now();
    return chrono::duration<double, milli>(e-s).count();
}

// -----
// MAIN
// -----

int main() {
    const int N = 10'000'000;

    // Generate random data
    vector<int> arr(N);
    mt19937 rng(99);
    uniform_int_distribution<int> dist(1, 1000000);
    for (auto& x : arr) x = dist(rng);

    cout << "===== " << endl;
    cout << " Parallel Reduction (Min/Max/Sum/Avg) via OpenMP " << endl;
    cout << " Array size: " << N << " elements " << endl;
    cout << "===== " << endl;
    cout << "Available threads: " << omp_get_max_threads() << endl;

    // — MIN —————
    int s_min, p_min;
    double t_s = timeIt([&]{ s_min = seq_min(arr); });

```

```

double t_p = timeIt([&]{ p_min = par_min(arr); });
cout << "\n--- MIN ---" << endl;
cout << " Sequential: " << s_min << " Time: " << t_s << " ms" << endl;
cout << " Parallel: " << p_min << " Time: " << t_p << " ms"
    << " Speedup: " << t_s/t_p << "x" << endl;
cout << " Match: " << (s_min == p_min ? "YES" : "NO") << endl;

// — MAX —————
int s_max, p_max;
t_s = timeIt([&]{ s_max = seq_max(arr); });
t_p = timeIt([&]{ p_max = par_max(arr); });
cout << "\n--- MAX ---" << endl;
cout << " Sequential: " << s_max << " Time: " << t_s << " ms" << endl;
cout << " Parallel: " << p_max << " Time: " << t_p << " ms"
    << " Speedup: " << t_s/t_p << "x" << endl;
cout << " Match: " << (s_max == p_max ? "YES" : "NO") << endl;

// — SUM —————
long long s_sum, p_sum;
t_s = timeIt([&]{ s_sum = seq_sum(arr); });
t_p = timeIt([&]{ p_sum = par_sum(arr); });
cout << "\n--- SUM ---" << endl;
cout << " Sequential: " << s_sum << " Time: " << t_s << " ms" << endl;
cout << " Parallel: " << p_sum << " Time: " << t_p << " ms"
    << " Speedup: " << t_s/t_p << "x" << endl;
cout << " Match: " << (s_sum == p_sum ? "YES" : "NO") << endl;

// — AVERAGE —————
double s_avg, p_avg;
t_s = timeIt([&]{ s_avg = seq_avg(arr); });
t_p = timeIt([&]{ p_avg = par_avg(arr); });
cout << "\n--- AVERAGE ---" << endl;
cout << " Sequential: " << s_avg << " Time: " << t_s << " ms" << endl;
cout << " Parallel: " << p_avg << " Time: " << t_p << " ms"
    << " Speedup: " << t_s/t_p << "x" << endl;
cout << " Match: " << (abs(s_avg - p_avg) < 1e-6 ? "YES" : "NO") << endl;

```

```
cout << "\n===== " << endl;
```

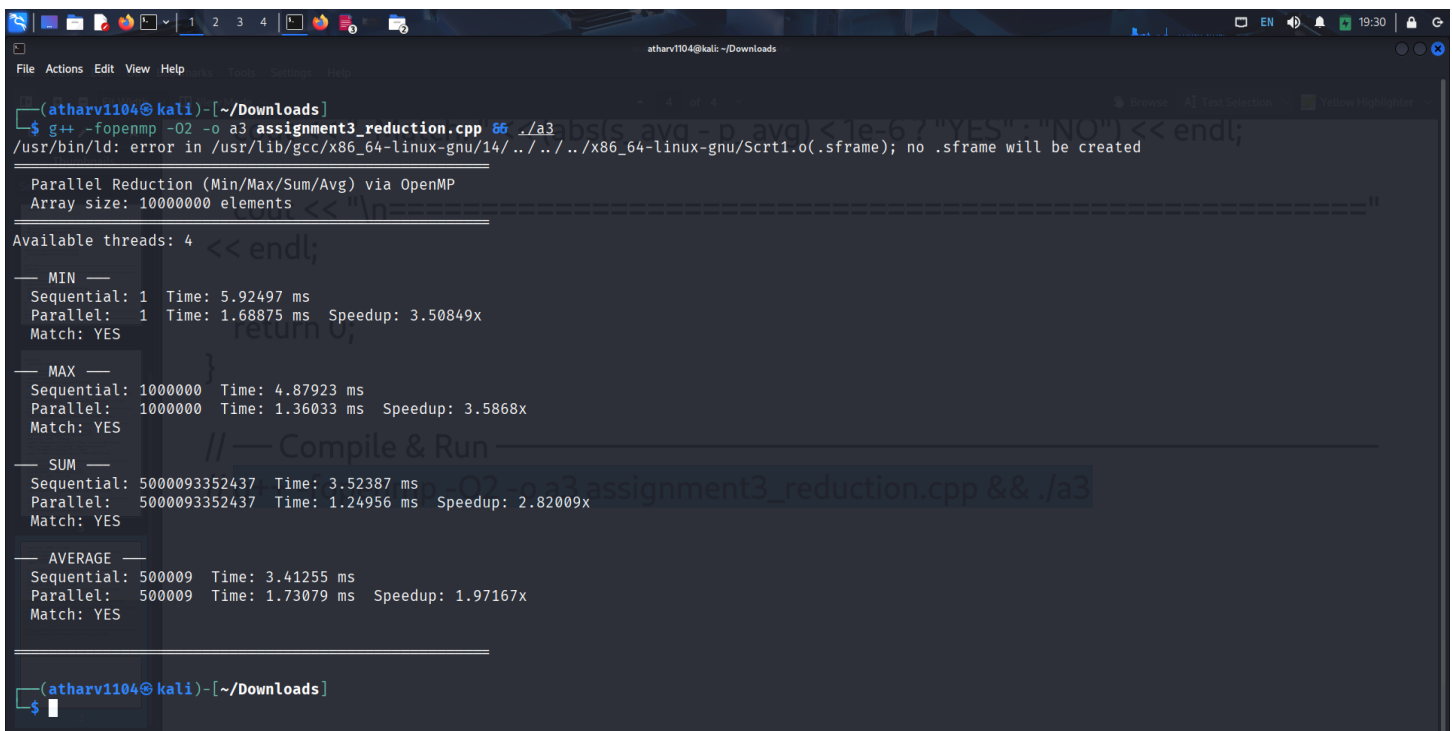
```
return 0;
```

```
}
```

```
// — Compile & Run —
```

```
// g++ -fopenmp -O2 -o a3 assignment3_reduction.cpp && ./a3
```

OUTPUT



```
(atharv1104@kali) ~/Downloads
$ g++ -fopenmp -O2 -o a3 assignment3_reduction.cpp && ./a3
/usr/bin/ld: error in /usr/lib/gcc/x86_64-linux-gnu/14/../../../../x86_64-linux-gnu/Scrt1.o(.sframe); no .sframe will be created

Parallel Reduction (Min/Max/Sum/Avg) via OpenMP
Array size: 10000000 elements

Available threads: 4

--- MIN ---
Sequential: 1 Time: 5.92497 ms
Parallel: 1 Time: 1.68875 ms Speedup: 3.50849x
Match: YES

--- MAX ---
Sequential: 1000000 Time: 4.87923 ms
Parallel: 1000000 Time: 1.36033 ms Speedup: 3.5868x
Match: YES

--- SUM ---
Sequential: 5000093352437 Time: 3.52387 ms
Parallel: 5000093352437 Time: 1.24956 ms Speedup: 2.82009x
Match: YES

--- AVERAGE ---
Sequential: 500009 Time: 3.41255 ms
Parallel: 500009 Time: 1.73079 ms Speedup: 1.97167x
Match: YES

(atharv1104@kali) ~/Downloads
$
```